

REQUISITOS PARA CRIAÇÃO DE UMA I.A PARA ANALISAR FEEDBACKS DE NOSSAS AULAS

1. ENTENDENDO O CONCEITO

Vamos entender o funcionamento de uma IA que identifica se uma mensagem é “positiva” ou “negativa” seguindo o ciclo completo:

dados → **treinamento** → **processamento** → **resultado** → **melhoria**.

Imagine que queremos criar uma IA capaz de analisar comentários de alunos e dizer se o comentário é positivo ou negativo.

1.1 Alimentação de dados

Comentário	Classificação
"A aula foi excelente!"	Positivo
"Gostei muito do projeto."	Positivo
"A explicação foi ótima."	Positivo
"Não gostei da aula."	Negativo
"O conteúdo foi muito difícil."	Negativo
"A aula foi ruim."	Negativo

Analogia: é como ensinar uma criança mostrando vários exemplos de coisas antes de pedir que ela reconheça algo sozinha.

1.2 Processamento

A IA transforma os textos em informações que o computador consegue analisar.

Por exemplo:

"A aula foi excelente!"

O sistema pode identificar palavras importantes como:

Aula, excelente, boa, gostei, ótima

E aprender que palavras como "excelente", "ótima" e "gostei" aparecem frequentemente em textos positivos.

Aqui entra o modelo de IA, que encontra padrões nos dados.

1.3 Treinamento

A IA recebe muitos exemplos e tenta encontrar uma regra/padrão.

De forma simplificada:

Dados de treinamento → Modelo de I.A procura padrões → Aprende a classificar

A IA não "entende" necessariamente o texto como uma pessoa. Ela aprende padrões a partir dos dados.

1.4 Entrada de um novo dado

Depois de treinada, podemos apresentar uma frase que ela nunca viu antes:

"Eu adorei essa aula!"

A frase é enviada para o modelo.

"Eu adorei essa aula!" → MODELO → Positivo

1.5 Execução / inferência

Nesse momento não estamos mais "ensinando" a IA.

Estamos usando o modelo que foi treinado.

Podemos representar:

Usuário → "Eu gostei muito!" → Processar modelo IA → Positivo → Resposta

Esse processo de utilizar o modelo treinado para produzir uma resposta é frequentemente chamado de inferência.

6. E se a IA errar?

Imagine que o aluno escreve:

"A aula não foi ruim."

A IA pode ter dificuldade porque encontra a palavra "ruim", mas a frase, no contexto, é positiva.

Então podemos:

- identificar o erro;
- adicionar novos exemplos aos dados;
- melhorar o processamento/modelo;
- treinar novamente;

- testar novamente.

Isso cria um ciclo:

Alimentação de dados → Processamento → Treinamento → Execução → Resultado → Avaliação → Se errou = Novos dados devem ser inseridos

Você pode resumir assim:

"Uma IA recebe dados, processa esses dados para encontrar padrões, aprende um modelo, recebe novos dados, executa o modelo e produz um resultado. Quando o resultado não é satisfatório, podemos melhorar os dados e o processo de treinamento."

Podemos transformar isso em um pequeno projeto:

Projeto: "IA Classificadora de Comentários"

Criar uma tabela com 50 comentários, classificando-os manualmente como:

- ✓ Positivo
- Negativo
- Neutro

Vamos começar com uma versão simples e didática, sem usar uma IA complexa: vamos criar um pequeno classificador baseado em exemplos, que depois pode evoluir para um modelo de Machine Learning.

A ideia será:

Frase → processamento → identificação de palavras/padrões → Positivo / Negativo / Neutro

Por exemplo:

- "A aula foi excelente!" → **Positivo**
- "Não gostei da aula." → **Negativo**
- "A aula foi hoje às 14 horas." → **Neutro**

2. CRIANDO A PRIMEIRA VERSÃO

2.1 Uma primeira versão em Python

Podemos usar um código bem simples, com listas de palavras positivas e negativas.

2.2 Mas existe um problema importante

Essa primeira versão **não é realmente uma IA treinada**. Ela é um classificador baseado em regras.

Isso pode ser ótimo para **os conceitos de IA**, porque podemos evoluir o projeto em etapas:

Versão 1 — Regras

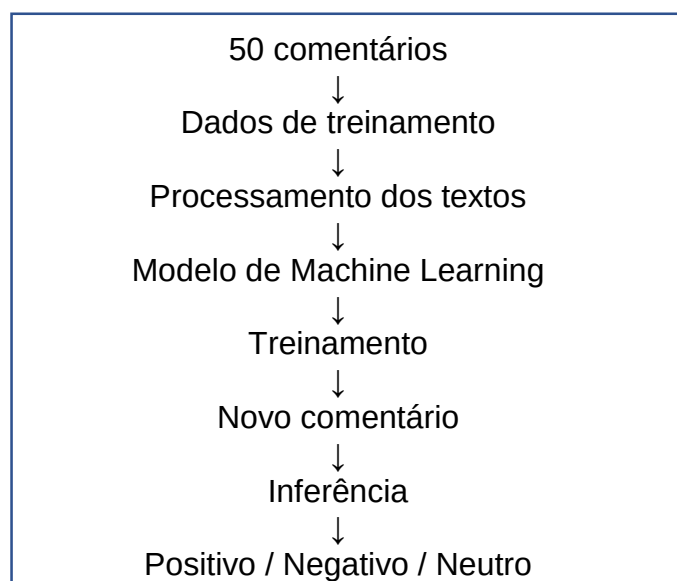
Frase → Procura palavras positivas/negativas → Conta pontos → Positivo / Negativo / Neutro → Classifica → retorna o resultado

Versão 2 — Dados de treinamento

Vamos inserir os dados de nossa planilha para treinar nossa I.A.

Versão 3 — Machine Learning

Podemos usar Python e bibliotecas como scikit-learn:



Dados → Processamento → Treinamento → Inferência → Resultado → Avaliação → Melhoria

Para continuar, podemos seguir a sequência:

Planilha → separar frases e classificações → transformar texto em números → treinar modelo → testar → classificar novas frases

2.3 Instalar as bibliotecas

No terminal:

pip install pandas scikit-learn

Vamos utilizar:

- **pandas** → para ler a planilha;
- **scikit-learn** → para transformar os textos e treinar o modelo.

2.4 O primeiro programa de IA

Podemos começar com este código: [ia_frases.py](#)

2.5 O que está acontecendo aqui?

Alimentação de dados

Primeiro:

```
dados = pd.read_csv("comentarios.csv")
```

O Python lê os 50 exemplos da planilha.

Temos:

Frase	Classificação
"A aula foi excelente!"	→ Positivo
"Gostei do projeto."	→ Positivo
"A aula foi ruim."	→ Negativo
"A aula foi normal."	→ Neutro

Isso corresponde à **alimentação de dados** do seu material.

SEPARAÇÃO

Depois fazemos:

```
X = dados["frase"]  
y = dados["classificacao"]
```

Aqui estamos dizendo:

X = aquilo que a IA vai analisar
y = aquilo que queremos que ela aprenda a responder

Então:

```
X  
↓  
"A aula foi excelente"  
"Gostei muito"  
"A aula foi ruim"
```

MODELO

```
y  
↓  
Positivo  
Positivo  
Negativo
```

2.6 Por que precisamos transformar texto em números?

O computador não consegue simplesmente entregar: "A aula foi excelente!" diretamente para o algoritmo de Machine Learning.

Precisamos transformar o texto em uma representação numérica.

É justamente o que fazemos aqui:

```
vetorizador = TfidfVectorizer()
```

E depois:

```
X_treino_numerico = vetorizador.fit_transform(X_treino)
```

Imagine, de forma simplificada, que temos:

```
"A aula foi excelente"  
↓
```

aula → 0.32
excelente → 0.71
foi → 0.00

A IA transforma o texto em números para que o algoritmo consiga encontrar padrões.

Isso corresponde à etapa de **Processamento** do seu material.

2.7 E onde acontece o treinamento?

Aqui:

```
modelo.fit(X_treino_numerico, y_treino)
```

Essa é uma linha extremamente importante.

"Agora estamos treinando o modelo."

O modelo recebe exemplos como:

Texto	Classificação
"A aula foi excelente"	→ Positivo
"Gostei muito da aula"	→ Positivo
"A aula foi ruim"	→ Negativo
"Não gostei da explicação"	→ Negativo
"A aula aconteceu hoje"	→ Neutro

E tenta encontrar padrões que relacionem os textos às classificações.

2.8 Por que separar treinamento e teste?

Essa parte é fundamental para explicar o conceito de **inferência**.

Temos 50 frases.

Podemos separar, por exemplo:

50 frases
↓
40 frases → TREINAMENTO
10 frases → TESTE

O código:

```
train_test_split(  
    X,  
    y,  
    test_size=0.2,
```

```
random_state=42
)
```

faz aproximadamente isso.

Então a IA **não vê as 10 frases de teste durante o treinamento.**

Depois perguntamos:

"Agora que você aprendeu com essas 40 frases, consegue classificar corretamente estas 10 que você nunca viu?"

Isso é muito mais interessante pedagogicamente do que testar a IA com as mesmas frases usadas no treinamento.

2.9 Como saber se ela acertou?

Temos:

```
previsoes = modelo.predict(X_teste_numerico)
```

O modelo faz suas previsões.

Por exemplo:

Resposta verdadeira IA respondeu

Positivo	Positivo	✓
Negativo	Negativo	✓
Neutro	Neutro	✓
Positivo	Negativo	X

Depois:

```
acuracia = accuracy_score(y_teste, previsoes)
```

Podemos obter algo como:

Acurácia: 0.80

Isso significa que, nesse conjunto de teste, o modelo acertou aproximadamente **80%** das classificações.

Com apenas 50 frases, porém, a acurácia pode variar bastante. **50 exemplos é um conjunto pequeno**, ótimo para um projeto didático, mas insuficiente para concluir que o modelo funcionaria bem em situações reais.

2.10 Finalmente: uma frase que a IA nunca viu

Essa é a parte que fecha o ciclo do seu material:

```
frase = ["Eu adorei essa aula!"]
```

Depois:

```
frase_numerica = vetorizador.transform(frase)
```

E:

```
resultado = modelo.predict(frase_numerica)
```

Podemos obter:

Frase: Eu adorei essa aula!

Classificação: Positivo

Isso é a **inferência**.

O modelo já foi treinado. Agora estamos apenas utilizando aquilo que ele aprendeu.

2.11 Podemos transformar isso em um projeto completo

PROJETO: IA CLASSIFICADORA DE AULAS

Comentários → Dados de treinamento → Processamento do texto → Treinamento do modelo → Dados de teste → Avaliação

Acertou → Resultado
Errou → Melhorar dados → Treinar novamente →

E depois podemos criar uma interface simples, onde o aluno digita:

Digite seu comentário:

> A explicação do professor foi muito boa!

E o programa responde:

> Analisando...

Classificação: POSITIVO